CSE 4631

Digital Signal Processing

Week 14

Filter basics

- Digital filters are used for two general purposes
  - **Signal separation:** Separation of a signal that has been contaminated with interference, noise, or other signals.
  - **Signal restoration:** Restoration of signals that have been distorted in some way.
- It is common in DSP to say that a filter's input and output signals are in the time domain because signals are usually created by sampling at regular intervals of time.
- However, the second most common way of sampling is at equal intervals of *space*.
  - Example: Taking simultaneous readings from an array of strain sensors mounted at one centimeter increments along the length of an aircraft wing.

Filter basics (contd.)

- Every linear filter has an **impulse response**, a **step response**, and a **frequency response**.
  - Each of these responses contains complete information about the filter, but in different form.
  - If one of them is specified, the other two are fixed and can be directly calculated.
  - All three of them are important, because they describe how the filter will react under different circumstances.
- The most straightforward way to implement a digital filter is by *convolving* the input signal with the digital filter's *impulse response*.
  - When the impulse response is used in this way, it is called **the filter kernel**.

Filter basics (contd.)

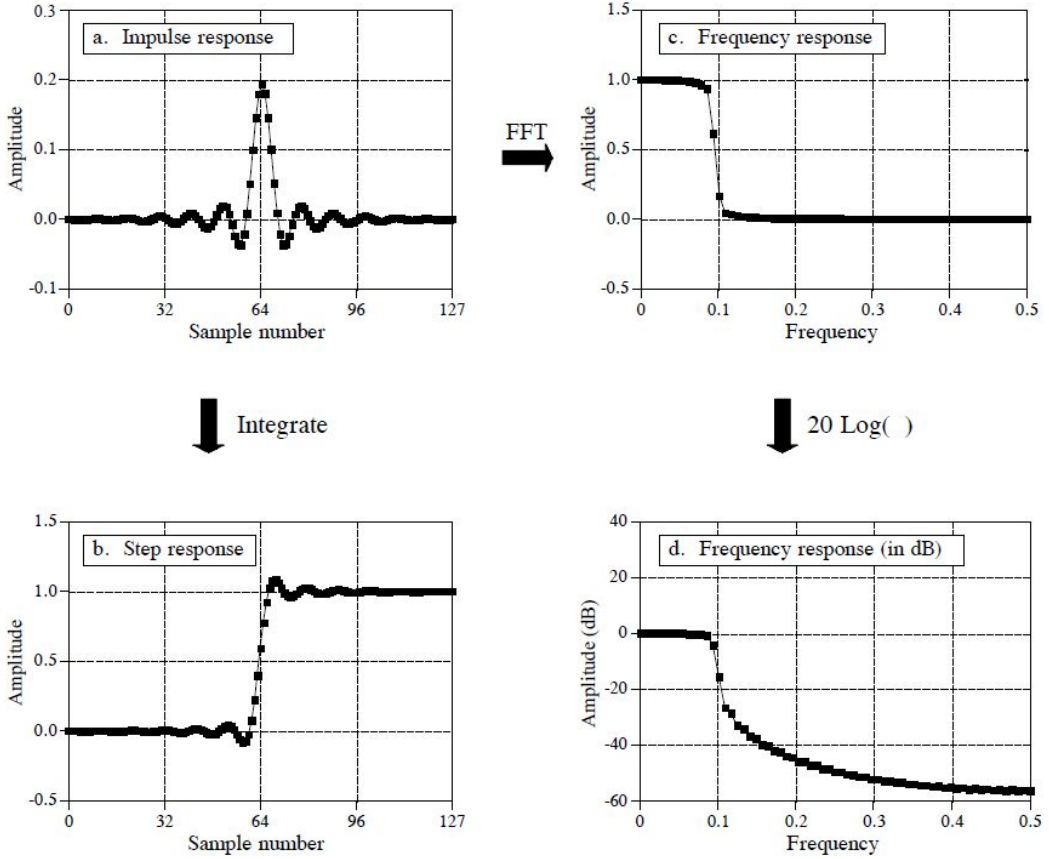


FIGURE 14-1
Filter parameters. Every linear filter has an impulse response, a step response, and a frequency response. The step response, (b), can be found by discrete integration of the impulse response, (a). The frequency response can be found from the impulse response by using the Fast Fourier Transform (FFT), and can be displayed either on a linear scale, (c), or in decibels, (d).

Filter basics (contd.)

- There is also another way to make digital filters, called **recursion**.
- When a filter is implemented by convolution, each sample in the output is calculated by weighting the samples in the input, and adding them together.
- Recursive filters are an extension of this, using previously calculated values from the *output*, besides points from the *input*.
- Instead of using a filter kernel, recursive filters are defined by a set of **recursion coefficients**.
- The impulse responses of recursive filters are composed of sinusoids that exponentially decay in amplitude. In principle, this makes their impulse responses *infinitely long*.
- However, the amplitude eventually drops below the round-off noise of the system, and the remaining samples can be ignored.
- Because of this characteristic, recursive filters are also called **Infinite Impulse Response or IIR** filters.
- In comparison, filters carried out by convolution are called **Finite Impulse Response or FIR** filters.

Filter basics (contd.)

- There are two ways to find the step response.
  - Feeding a step waveform into the filter and see what comes out
  - Integrate the impulse response – using *running sum*
- The frequency response can be plotted in two ways
  - On a linear vertical axis
  - On a logarithmic scale (decibels)
    - **1 bel means the power is changed by a factor of ten**
    - 3 bel means the power is changed by 3 factors of ten $(10\times10\times10 = 10^3)$
    - **1 decibel is one-tenth of a bel** (Therefore, $10\text{dB} = 1\text{B}$)
    - 10 decibels means that the power has changed by a factor of ten
    - 30 decibel means the power is changed by 3 factors of ten $(10\times10\times10 = 10^3 = 10^{30/10})$

Filter basics (contd.)

- But we want to work with a signal's amplitude, not its power
- Remember from Week 2, that amplitude is proportional to the square-root of power

$$10^{B} = \frac{P_2}{P_1} \qquad \sqrt{10^{B}} = \sqrt{\frac{P_2}{P_1}}$$

$$\Rightarrow 10^{\frac{dB}{10}} = \frac{P_2}{P_1} \qquad \Rightarrow \sqrt{10^{\frac{dB}{10}}} = \frac{A_2}{A_1}$$

$$\Rightarrow \frac{dB}{10} = \log_{10}\frac{P_2}{P_1} \qquad \Rightarrow 10^{\frac{dB}{20}} = \frac{A_2}{A_1}$$

$$\Rightarrow dB = 10\log_{10}\frac{P_2}{P_1} \qquad \Rightarrow dB = 20\log_{10}\frac{A_2}{A_1}$$

$$\Rightarrow dB = 4.342945\ln\frac{P_2}{P_1} \qquad \Rightarrow dB = 8.685890\ln\frac{A_2}{A_1}$$

Filter basics (contd.)

Some common conversions
between decibels and amplitude ratios:

$$
\begin{aligned}
60\text{dB} &= 1000 \\
40\text{dB} &= 100 \\
20\text{dB} &= 10 \\
0\text{dB} &= 1 \\
-20\text{dB} &= 0.1 \\
-40\text{dB} &= 0.01 \\
-60\text{dB} &= 0.001
\end{aligned}
$$

Filter basics (contd.)

- Since decibels are a way of expressing the ratio between two signals, they are ideal for describing the gain of a system (ratio between the output and input signal)
- Decibels are also used to specify the amplitude (or power) of a single signal, by referencing it to some standard. For example,
  - The term: **dBV** means that the signal is being referenced to a 1 volt rms signal.
  - Likewise, **dBm** indicates a reference signal producing 1 mW into a 600 ohms load (about 0.78 volts rms).

How Information is Represented in Signals

- There are only **two ways** that are common for information to be represented in naturally occurring signals:
  - Information represented in the time domain
    - Describes when something occurs and what the amplitude of the occurrence is
    - Even if you have only one sample from this signal, you still know something about what you are measuring.
    - Simples way for information to be contained in a signal
  - Information represented in the frequency domain.
    - Contains information about a system producing a periodic motion by measuring the frequency, phase, and amplitude of the motion.
    - A single sample, in itself, contains no information about the periodic motion, and therefore no information about the system
    - The information is contained in the relationship between many points in the signal.

How Information is Represented in Signals (contd.)

- The step response describes how information represented in the time domain is being modified by the system.
- In contrast, the frequency response shows how information represented in the frequency domain is being changed.
- **Good performance in the time domain results in poor performance in the frequency domain, and vice versa.**
    - It is not possible to optimize a filter for both applications
- If you are designing a filter to remove noise from an EKG signal (information represented in the time domain), the step response is the important parameter, and the frequency response is of little concern.
- If your task is to design a digital filter for a hearing aid (with the information in the frequency domain), the frequency response is all important, while the step response doesn't matter.

Time Domain Parameters

Wondering why the step response is the important parameter in time domain filters over the impulse response?

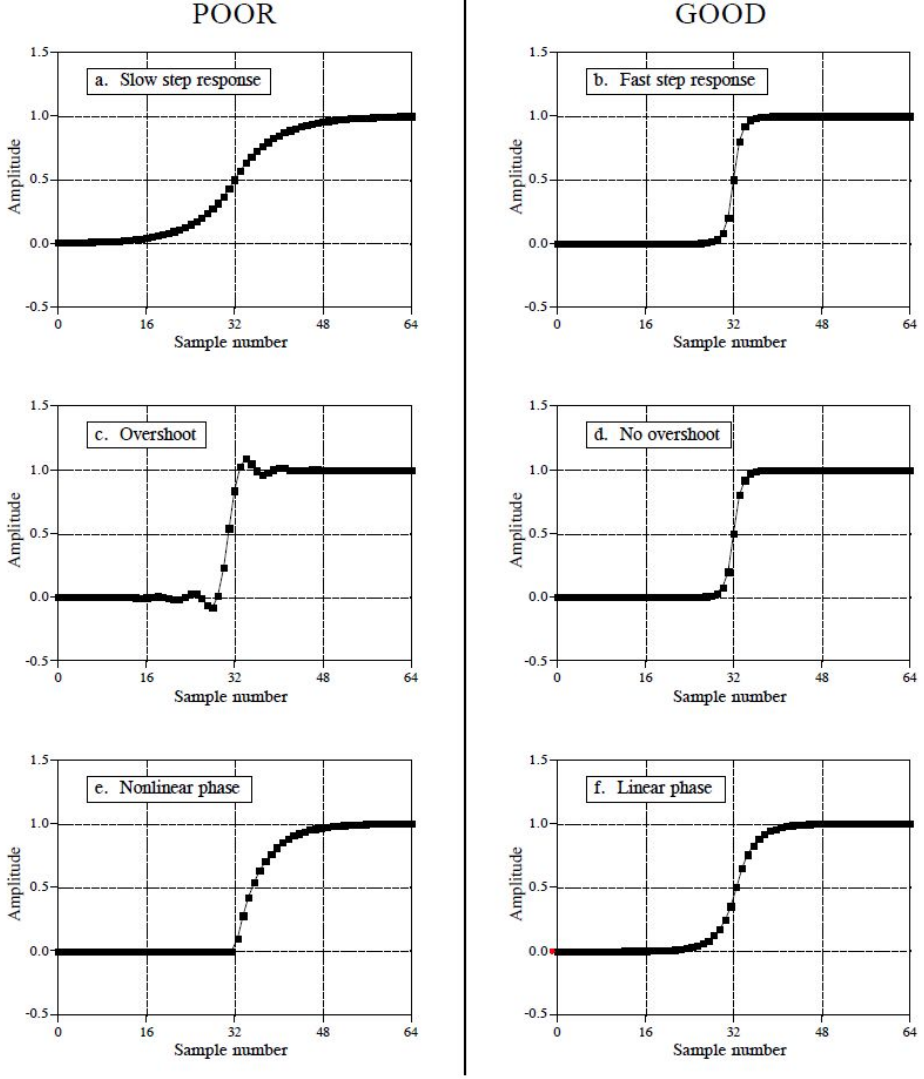
- Because it matches the way human minds understands and processes information.
- The first thing humans do while analyzing a random signal of some unknown origin is to divide the signal into regions of similar characteristics.
- This segmentation is accomplished by identifying the points that separate the regions.
- The step function is the purest way of representing a division between two dissimilar regions.
    - It can mark when an event starts, or when an event ends. It tells you that whatever is on the left is somehow different from whatever is on the right.
- The step response, in turn, is important because it describes how the dividing lines are being modified by the filter.

Time Domain Parameters (contd.)

Step response parameters: Risetime

- To distinguish events in a signal, the duration of the step response must be shorter than the spacing of the events. Meaning that the step response should be as *fast* as possible.
- **Risetime** is a way to measure how fast a system responds to a sudden change.
- Imagine you feed the system a step signal that jumps from 0 to some maximum amplitude instantly. In reality, no physical system can respond instantly — it will gradually rise toward the final value.
- Risetime is the time it takes for the response to go from 10% of its final value to 90% of its final value.
- In digital systems, instead of measuring in seconds, we often measure how many samples it takes for that rise to happen.

Time Domain Parameters (contd.)



Time Domain Parameters (contd.)

Step response parameters: Overshoot

- Overshoot must generally be eliminated because it changes the amplitude of samples in the signal; this is a basic distortion of the information contained in the time domain.

Step response parameters: Linearity of the phase

- It is often desired that the upper half of the step response be symmetrical with the lower half.
- This symmetry is needed to make the rising edges look the same as the falling edges.
- This symmetry is called **linear phase**, because the frequency response has a phase that is a straight line

Frequency Domain Parameters

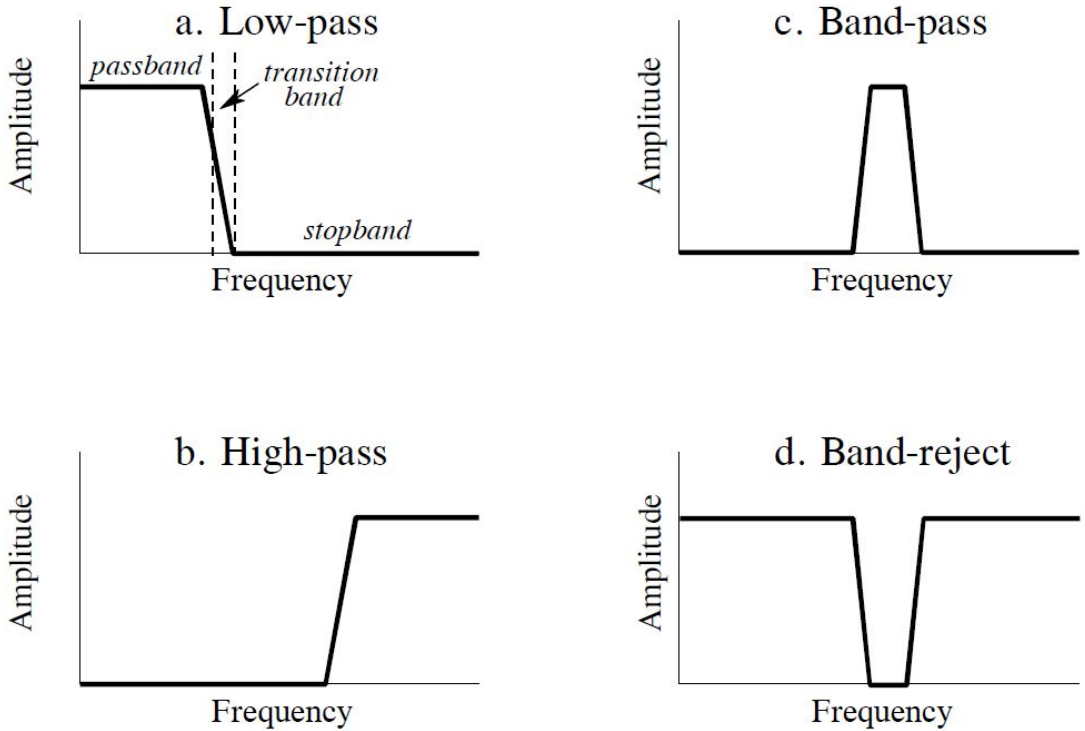
There are four basic frequency responses: **Low-pass**, **high-pass, band-pass**, and **band-reject**.

The purpose of these filters is to allow some frequencies to pass unaltered while completely blocking other frequencies. Each of them has the following components:

- The **passband** refers to those frequencies that are passed.
- The **stopband** contains those frequencies that are blocked.
- The **transition band** is in between.
- A **fast roll-off** means that the transition band is very narrow.
- The division between the passband and transition band is called the **cutoff frequency**.
    - In analog filter design, the cutoff frequency is usually defined to be where the amplitude is reduced to 0.707 (i.e., -3dB).
    - Digital filters are less standardized, and it is common to see 99%, 90%, 70.7%, and 50% amplitude levels defined to be the cutoff frequency.

Frequency Domain Parameters (contd.)

FIGURE 14-3
The four common frequency responses. Frequency domain filters are generally used to pass certain frequencies (the *passband*), while blocking others (the *stopband*). Four responses are the most common: low-pass, high-pass, band-pass, and band-reject.

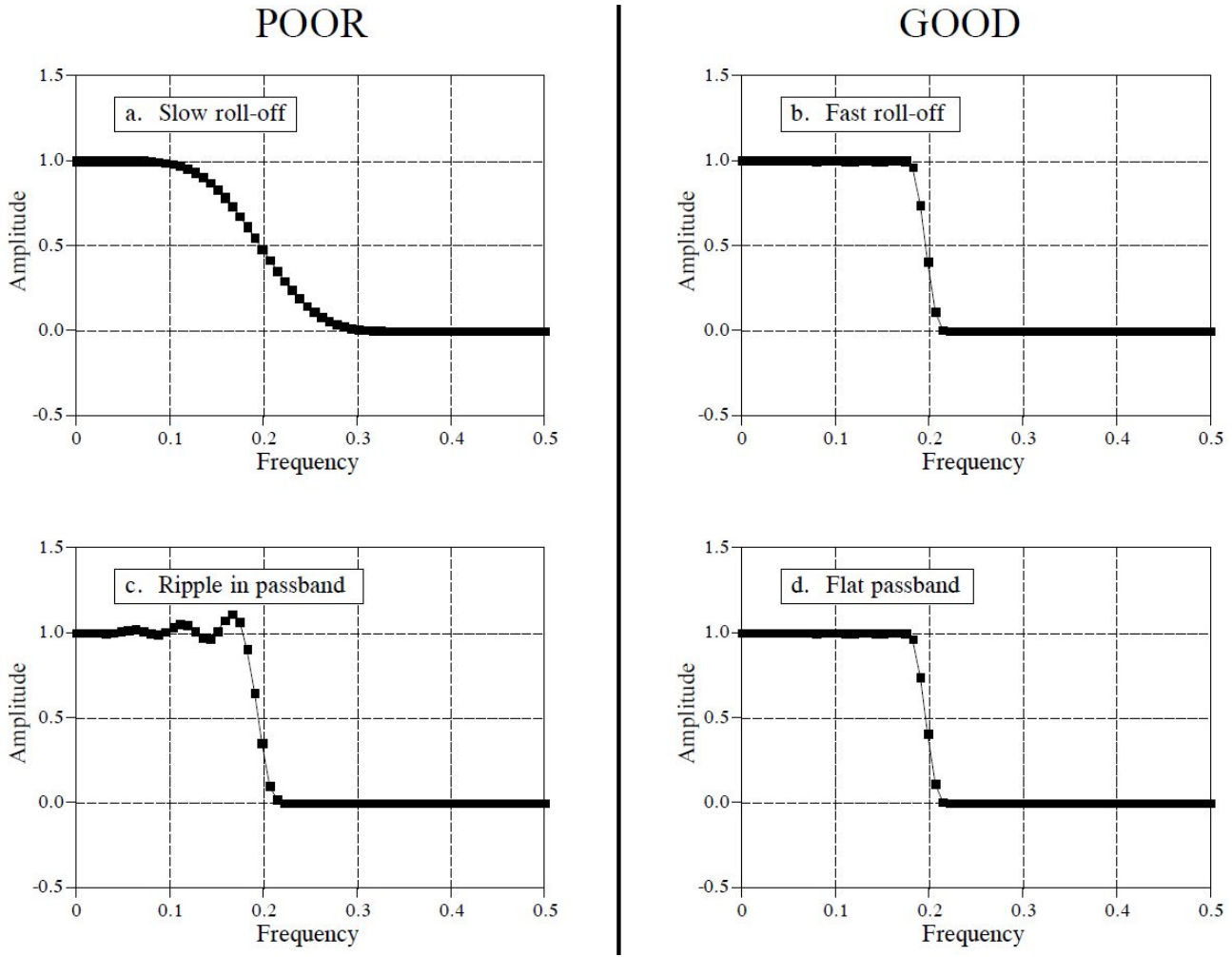


Frequency Domain Parameters (contd.)

There are three parameters that measure how well a filter performs in the frequency domain:

1. **Roll-off speed**
    - To separate closely spaced frequencies, the filter must have a fast roll-off.
2. **Passband ripple**
    - For the passband frequencies to move through the filter unaltered, there must be no passband ripple.
3. **Stopband attenuation**
    - To adequately block the stopband frequencies, it is necessary to have good stopband attenuation.

Frequency Domain Parameters (contd.)



Frequency Domain Parameters (contd.)

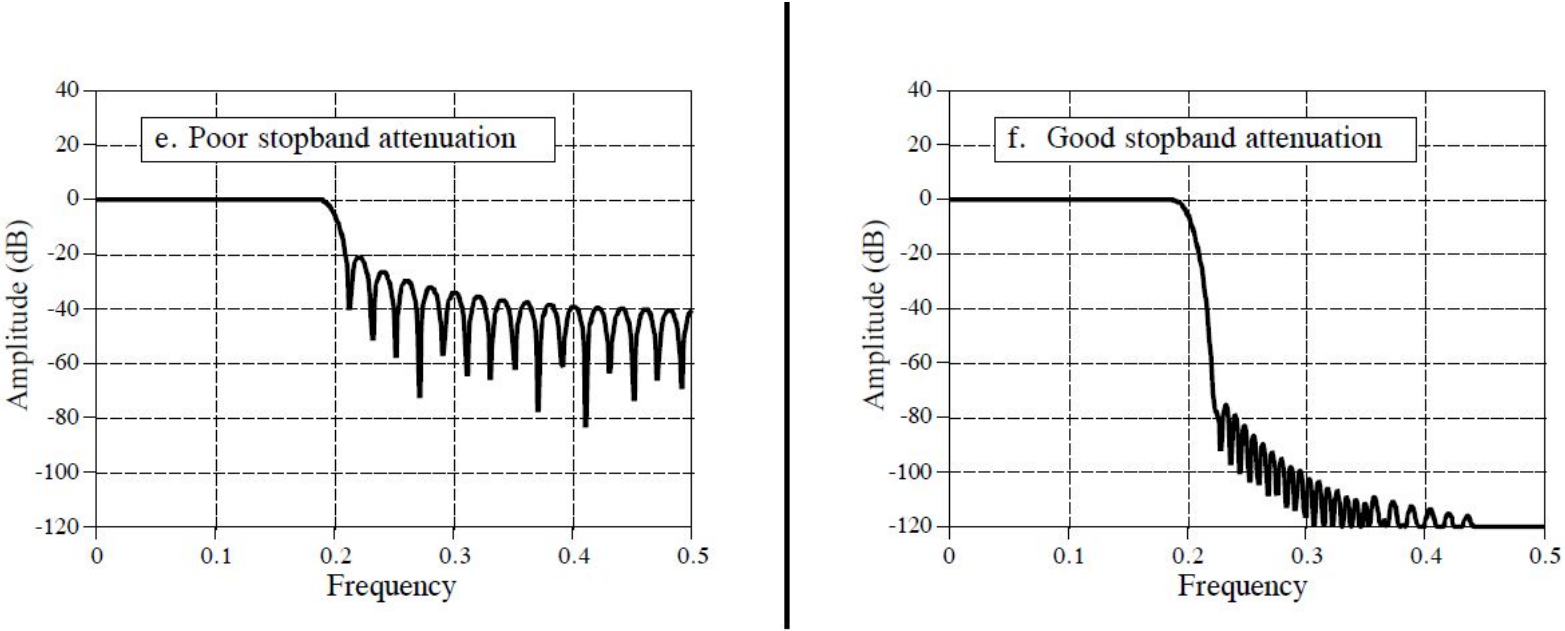


FIGURE 14-4
Parameters for evaluating *frequency domain* performance. The frequency responses shown are for low-pass filters. Three parameters are important: (1) roll-off sharpness, shown in (a) and (b), (2) passband ripple, shown in (c) and (d), and (3) stopband attenuation, shown in (e) and (f).

High-Pass, Band-Pass and Band-Reject Filters

- These filters are designed by starting with a low-pass filter, and then converting it into the desired response.
- There are two methods for the low-pass to high-pass conversion (changing the low-pass filter kernel into a high-pass filter kernel)
- **Method 1: Spectral inversion**
  - **Step 1:** Change the sign of each sample in the filter kernel
  - **Step 2:** Add *one* to the sample at the center of the symmetry
  - *Flips the frequency response top-for-bottom*, changing the passband into stopbands, and the stopbands into passbands
  - Changes a filter from low-pass to high-pass, high-pass to low-pass, band-pass to band-reject, or band-reject to band-pass.

High-Pass, Band-Pass and Band-Reject Filters (contd.)

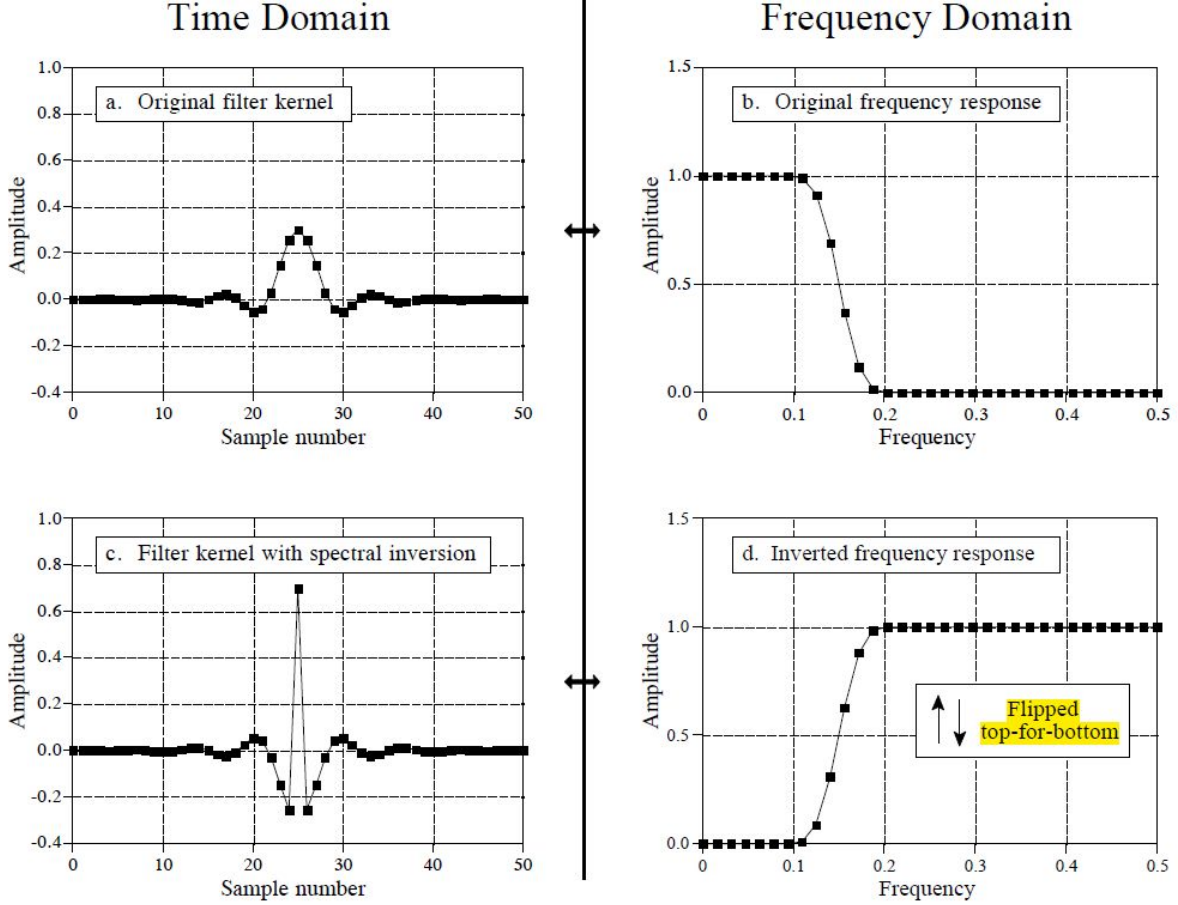


FIGURE 14-5
Example of spectral inversion. The low-pass filter kernel in (a) has the frequency response shown in (b). A high-pass filter kernel, (c), is formed by changing the sign of each sample in (a), and adding one to the sample at the center of symmetry. This action in the time domain *inverts* the frequency spectrum (i.e., flips it top-for-bottom), as shown by the high-pass frequency response in (d).

High-Pass, Band-Pass and Band-Reject Filters (contd.)

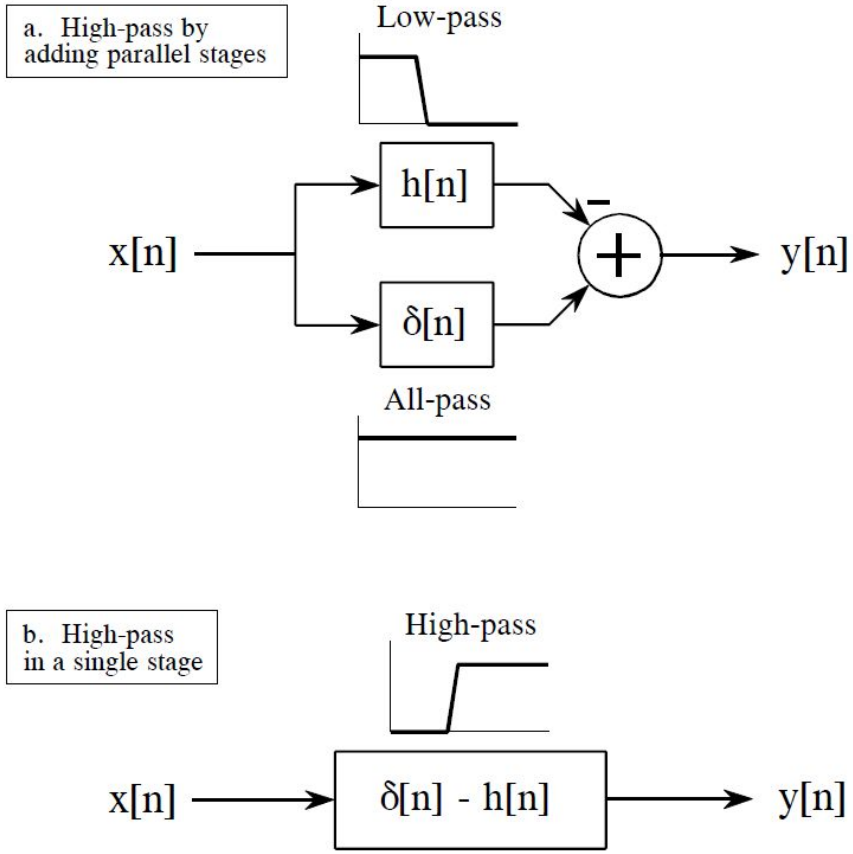


FIGURE 14-6
Block diagram of spectral inversion. In (a), the input signal, $x[n]$, is applied to two systems in parallel, having impulse responses of $h[n]$ and $\delta[n]$. As shown in (b), the combined system has an impulse response of $\delta[n] - h[n]$. This means that the frequency response of the combined system is the *inversion* of the frequency response of $h[n]$.

High-Pass, Band-Pass and Band-Reject Filters

- **Method 2 for low-pass to high-pass conversion: Spectral Reversal**
  - **Change the sign of every other sample in the filter kernel**
  - Equivalent to multiplying the filter kernel by a sinusoid with a frequency of 0.5
  - This has the effect of shifting the frequency domain by 0.5 (see week 13)
  - The negative frequencies between -0.5 and 0 that are of mirror images of the frequencies between 0 and 0.5 have shifted to the right by 0.5
  - *Flips the frequency response left-for-right*

Forming band-reject filter

- Adding the filter kernels of the low-pass and high-pass filters

Forming band-pass filter

- Convolving the filter kernels of the low-pass and high-pass filters

High-Pass, Band-Pass and Band-Reject Filters (contd.)

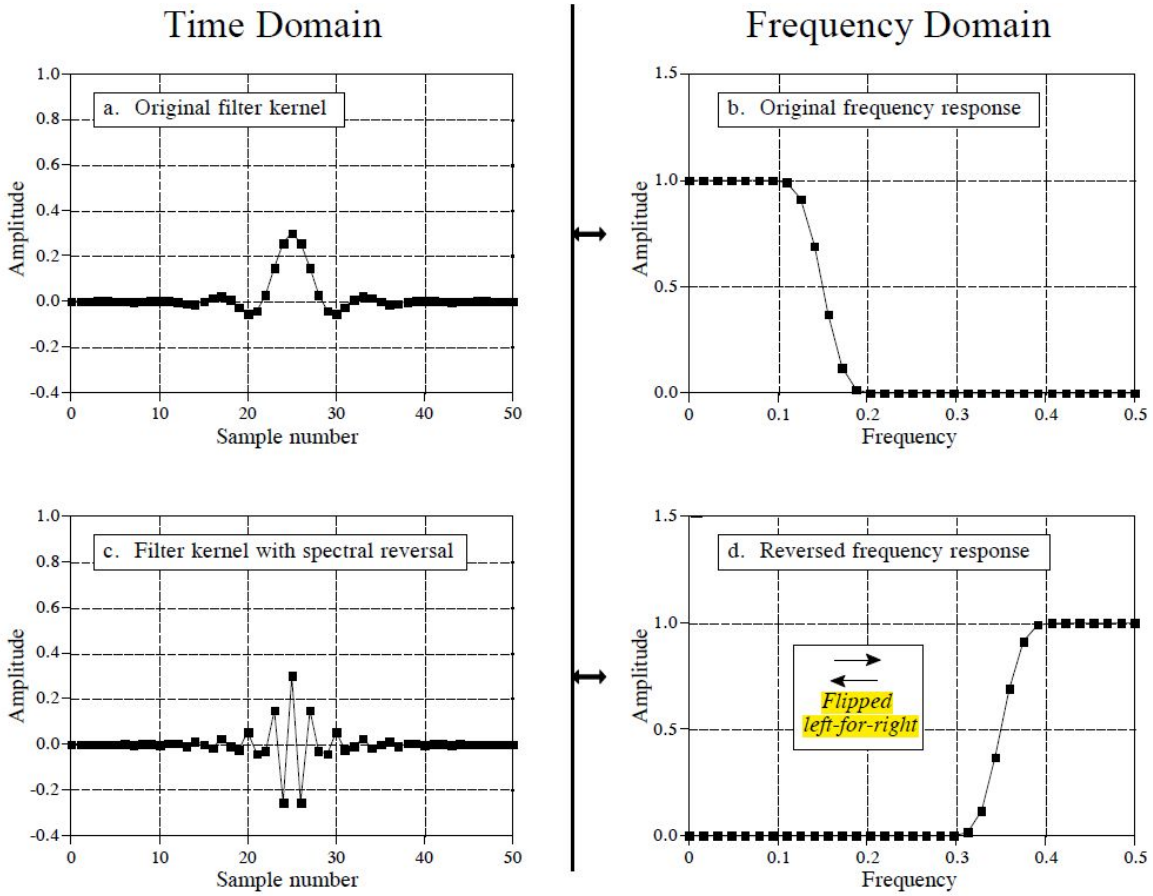


FIGURE 14-7
Example of spectral reversal. The low-pass filter kernel in (a) has the frequency response shown in (b). A high-pass filter kernel, (c), is formed by changing the sign of every other sample in (a). This action in the time domain results in the frequency domain being flipped *left-for-right*, resulting in the high-pass frequency response shown in (d).

High-Pass, Band-Pass and Band-Reject Filters (contd.)

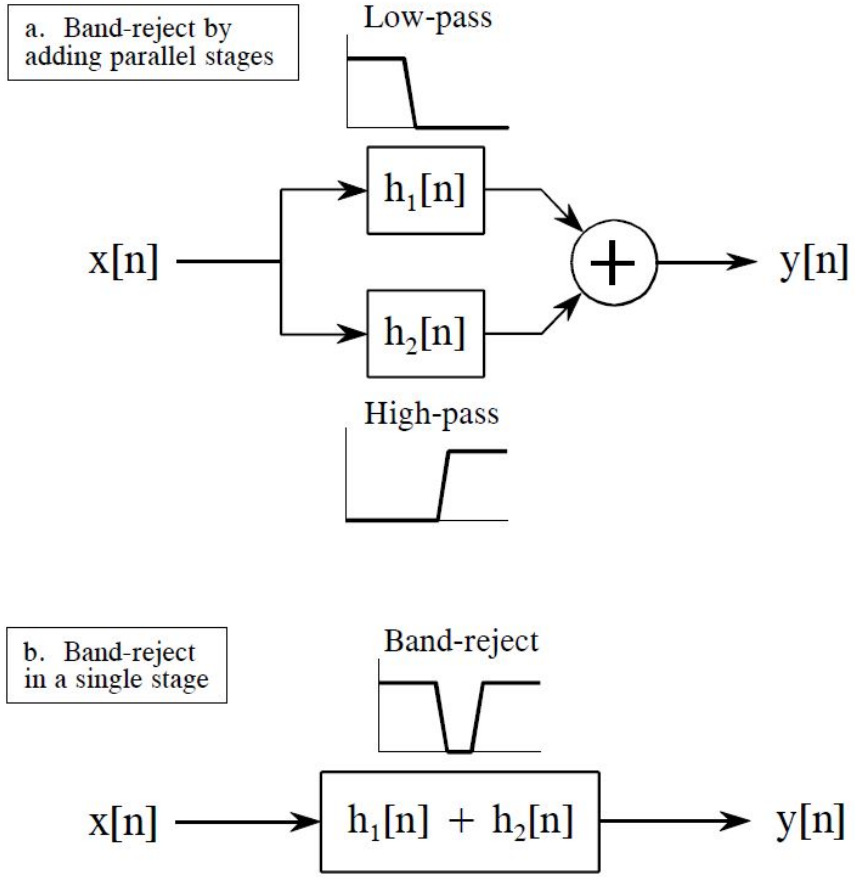


FIGURE 14-9
Designing a band-reject filter. As shown in (a), a band-reject filter is formed by the parallel combination of a low-pass filter and a high-pass filter with their outputs added. Figure (b) shows this reduced to a single stage, with the filter kernel found by *adding* the low-pass and high-pass filter kernels.

High-Pass, Band-Pass and Band-Reject Filters (contd.)

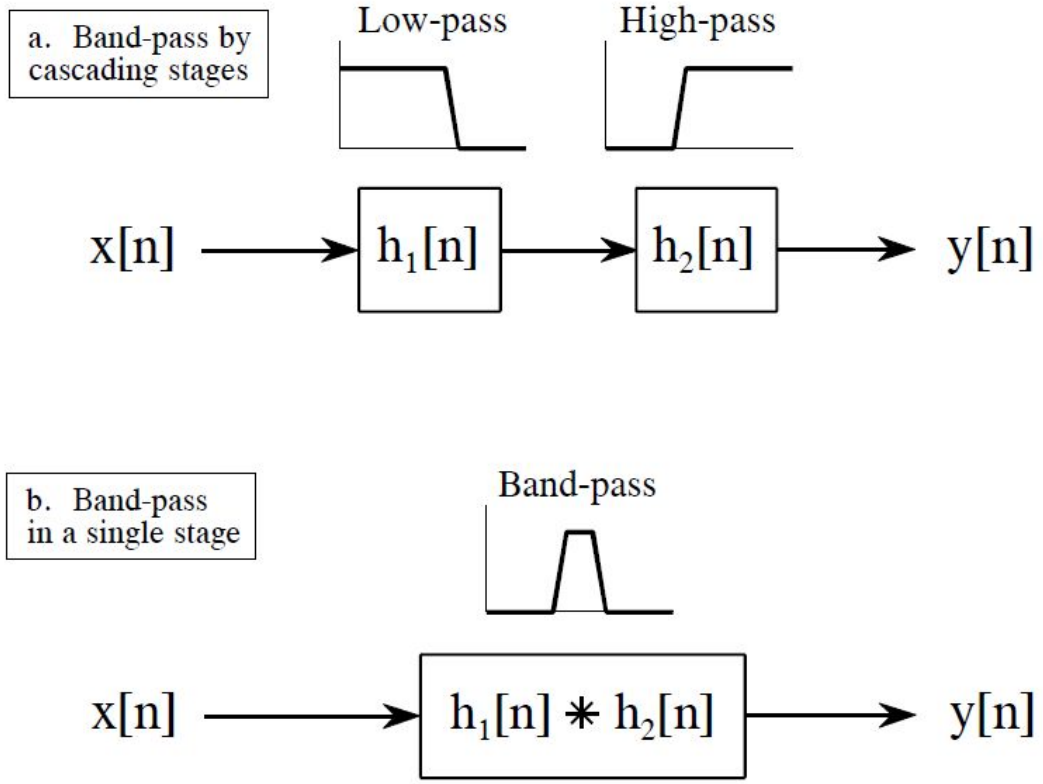


FIGURE 14-8
Designing a band-pass filter. As shown in (a), a band-pass filter can be formed by cascading a low-pass filter and a high-pass filter. This can be reduced to a single stage, shown in (b). The filter kernel of the single stage is equal to the *convolution* of the low-pass and high-pass filter kernels.

Filter Classification

The use of a digital filter can be broken into three categories: time domain, frequency domain and custom.

- **Time domain** filters are used when the information is encoded in the shape of the signal's waveform.
    - Example usages: smoothing, DC removal, waveform shaping, etc.
- **Frequency domain** filters are used when the information is contained in the amplitude, frequency, and phase of the component sinusoids.
- **Custom** filters are used when a special action is required by the filter, something more elaborate than the four basic responses

The use of a digital filter can be implemented in two ways: by convolution (also called FIR) and by recursion (also called IIR)

- Filters carried out by convolution can have far better performance than filters using recursion, but execute much more slowly.

Filter Classification

FILTER USED FOR:	FILTER IMPLEMENTED BY: Convolution *Finite Impulse Response (FIR)*	Recursion *Infinite Impulse Response (IIR)*
Time Domain *(smoothing, DC removal)*	Moving average (Ch. 15)	Single pole (Ch. 19)
Frequency Domain *(separating frequencies)*	Windowed-sinc (Ch. 16)	Chebyshev (Ch. 20)
Custom *(Deconvolution)*	FIR custom (Ch. 17)	Iterative design (Ch. 26)

TABLE 14-1
Filter classification. Filters can be divided by their *use*, and how they are *implemented*.